Session 5: SciPy & Image Processing

ESS: Programming with Python

Abate Alemayehu

Ethiopian Statistical Service (ESS)

August 2–15, 2025

Slide 341: What is SciPy?

- Scientific Python
- Built on NumPy
- Collection of scientific algorithms
- Open source, community-driven

Info

SciPy is the fundamental library for scientific computing in Python.

Slide 342: SciPy vs NumPy

NumPy

- Arrays
- Basic math
- Linear algebra

SciPy

- Advanced scientific computing
- Uses NumPy arrays
- Together: complete scientific stack

Slide 343: Installing and Importing

Installation

- Included in Anaconda
- `conda install scipy` or `pip install scipy`

Import Subpackages

```
1 from scipy import linalg
2 from scipy import optimize
3 from scipy import integrate
4 from scipy import sparse
```

Slide 344: SciPy Subpackages

Subpackage	Purpose
`scipy.cluster`	Clustering algorithms
`scipy.constants`	Physical constants
`scipy.fft`	Fourier transforms
`scipy.integrate`	Numerical integration
`scipy.interpolate`	Interpolation
`scipy.linalg`	Linear algebra
`scipy.optimize`	Optimization
`scipy.signal`	Signal processing
`scipy.sparse`	Sparse matrices
`scipy.stats`	Statistics
`scipy.ndimage`	Image processing

Slide 345: Physical Constants

```
1 from scipy import constants
2
3 print(constants.pi)           # pi
4 print(constants.c)            # speed of light
5 print(constants.h)            # Planck constant
6 print(constants.e)            # electron charge
7 print(constants.g)            # gravitational acceleration
```

Info

Constants are updated to CODATA recommended values.

Slide 346: Linear Algebra with scipy.linalg

- `scipy.linalg.inv(A)` → matrix inverse
- `scipy.linalg.det(A)` → determinant
- `scipy.linalg.eig(A)` → eigenvalues/vectors
- `scipy.linalg.svd(A)` → singular value decomposition

Info

More robust than `np.linalg`.

Slide 347: Solving Linear Systems

```
from scipy.linalg import solve

# Solve Ax = b
A = [[3, 1], [1, 2]]
b = [9, 8]
x = solve(A, b)
print(x)  # [2.0, 3.0]
```

Info

Faster and more stable than `inv(A) @ b`.

Slide 348: Numerical Integration

```
from scipy import integrate

# Definite integral
result, error = integrate.quad(func, a, b)

# Double integral
result, error = integrate.dblquad(func, a, b, gfun, hfun)
```

- Returns (result, error_estimate)
- Handles infinite limits ($-\infty$, ∞)

Slide 349: Integration Example

```
from scipy import integrate
import numpy as np

def f(x):
    return np.exp(-x**2)

result, error = integrate.quad(f, -np.inf, np.inf)
print(f"Integral = {result:.6f}")
# Output: 1.772454 (sqrt(pi))
```

Slide 350: Interpolation

```
from scipy import interpolate

# Create interpolation function
f = interpolate.interp1d(x, y, kind='linear')
f = interpolate.interp1d(x, y, kind='cubic')

# Evaluate at new points
y_new = f(x_new)
```

- `kind='linear'` → linear interpolation
- `kind='cubic'` → cubic spline

Slide 351: Optimization – Root Finding

- `optimize.root(func, x0)` → find root near x0
- `optimize.brentq(func, a, b)` → root in interval
- `optimize.fsolve(func, x0)` → for systems

Info

Brent's method is robust for 1D root finding.

Slide 352: Optimization – Minimization

- `optimize.minimize_scalar(func)` → 1D minimization
- `optimize.minimize(func, x0)` → multi-dimensional
- Methods: `method='BFGS'`, `method='Nelder-Mead'`

Slide 353: Curve Fitting

```
from scipy import optimize

def model_func(x, a, b):
    return a * x + b

params, cov = optimize.curve_fit(model_func, x_data, y_data, p0
    )
```

- Returns optimal parameters and covariance
- Example: fitting exponential decay, polynomial, etc.

Slide 354: Statistical Functions

- `from scipy import stats`
- Probability distributions: `stats.norm`, `stats.poisson`
- `stats.ttest_ind()` → t-test
- `stats.pearsonr()` → correlation
- `stats.chi2_contingency()` → chi-square test

Slide 355: Code Example: SciPy Basics I

```
import numpy as np
from scipy import integrate, optimize, stats

# Integration
def f(x):
    return x**2
area, _ = integrate.quad(f, 0, 1)
print(f"Integral of x^2 from 0 to 1 = {area}")

# Curve fitting
x = np.array([0, 1, 2, 3, 4])
y = np.array([1, 2.5, 3.8, 5.1, 6.2])

def linear(x, a, b):
```

Slide 355: Code Example: SciPy Basics II

```
    return a * x + b

params, _ = optimize.curve_fit(linear, x, y)
print(f"Slope: {params[0]:.2f}, Intercept: {params[1]:.2f}")

# Statistics
data = np.random.normal(0, 1, 1000)
p_value = stats.normaltest(data).pvalue
print(f"p-value for normality: {p_value:.4f}")
```

Slide 356: What are Sparse Arrays?

- Arrays where most elements are zero
- Store only non-zero values
- Save memory and computation time
- Common in: graphs, NLP, images

Slide 357: When to Use Sparse

- Density $< 10\%$ (few non-zeros)
- Very large arrays ($1000 \times 1000+$)
- Memory constraints
- Example: adjacency matrices, one-hot encoding

Slide 358: Sparse Formats in SciPy

Format	Best For	Storage
CSR	Arithmetic, row slicing	Compressed Sparse Row
CSC	Column slicing	Compressed Sparse Column
COO	Construction	Coordinate list
LIL	Incremental building	List of Lists
DOK	Dictionary access	Dictionary of Keys

Slide 359: CSR Format (Compressed Sparse Row)

- Most common for computations
- Efficient for row slicing
- Efficient for matrix-vector products
- Uses `data`, `indices`, `indptr` arrays

Slide 360: Creating Sparse Arrays

```
from scipy.sparse import csr_matrix, coo_matrix

# From dense array
dense = np.array([[0, 0, 1], [2, 0, 0], [0, 3, 0]])
sparse = csr_matrix(dense)

# From COO format
row = [0, 1, 2]
col = [2, 0, 1]
data = [1, 2, 3]
coo = coo_matrix((data, (row, col)), shape=(3, 3))
```

Slide 361: LIL Format for Building

```
from scipy.sparse import lil_matrix

# Best for incrementally building
lil = lil_matrix((1000, 1000))
lil[0, 500] = 5
lil[10, 20] = 3

# Convert to CSR for computation
csr = lil.tocsr()
```

Slide 362: DOK Format for Dictionary Access

```
from scipy.sparse import dok_matrix

# Like a dictionary: (row, col) -> value
dok = dok_matrix((100, 100))
dok[5, 5] = 10
dok[10, 20] = 30
```

- Good for random access
- Slower for arithmetic

Slide 363: Sparse Matrix Properties

- `sparse.shape` → dimensions
- `sparse.nnz` → number of non-zero entries
- `sparse.dtype` → data type
- `sparse.format` → storage format

Slide 364: Converting Between Formats

```
sparse.tocsr()    # to CSR
sparse.tocsc()    # to CSC
sparse.tocoo()    # to COO
sparse.todense() # to dense NumPy array
sparse.toarray() # same as todense()
```

Warning

Avoid converting large sparse matrices to dense!

Slide 365: Sparse Arithmetic

```
# Addition
result = sparse1 + sparse2

# Multiplication
product = sparse1 @ sparse2

# Element-wise
elementwise = sparse.multiply(sparse2)

# Scalar operations
scaled = sparse * 2
```

Slide 366: Sparse Matrix-Vector Multiplication

- Very efficient (O(nnz) time)
- `sparse @ vector`
- Used in iterative solvers
- Example: PageRank algorithm

Slide 367: Saving and Loading Sparse Matrices

```
from scipy.sparse import save_npz, load_npz

# Save
save_npz('matrix.npz', sparse)

# Load
loaded = load_npz('matrix.npz')
```

Info

Efficient binary format that preserves sparsity.

Slide 368: Eye and Identity

```
from scipy.sparse import eye, identity, diags

# Identity matrix
I = eye(1000, format='csr')
I = identity(1000, format='csr')

# Diagonal sparse matrix
diagonals = [1, 2, 3]
offsets = [0, 1, -1]
diag = diags(diagonals, offsets)
```

Slide 369: Random Sparse Matrices

```
from scipy.sparse import random

# 5% non-zero density
sparse = random(100, 100, density=0.05, format='csr')

# Custom distribution for values
def custom_rvs(size):
    return np.random.poisson(3, size)

sparse = random(100, 100, density=0.05, data_rvs=custom_rvs)
```

Slide 370: Sparse Linear Algebra

- `from scipy.sparse.linalg import spsolve`
- `spsolve(A, b)` $\rightarrow$ solve Ax = b (sparse A)
- `eigs(A, k=6)` $\rightarrow$ top 6 eigenvalues
- `svds(A, k=10)` $\rightarrow$ top 10 singular values

Slide 371: Iterative Solvers for Sparse Systems

- `from scipy.sparse.linalg import bicg, gmres, cg`
- Conjugate Gradient (symmetric positive definite)
- GMRES (general matrices)
- No need to store full matrix

Slide 372: Memory Comparison

Size (dense)	Density	Dense Memory	Sparse Memory
10,000 × 10,000	1%	800 MB	8 MB
100,000 × 100,000	0.1%	80 GB	80 MB
1M × 1M	0.01%	8 TB	80 MB

Slide 373: Performance Considerations

- CSR: good for arithmetic, row access
- CSC: good for column access
- COO: good for construction, conversion
- **Avoid:** dense conversion of large matrices

Slide 374: When NOT to Use Sparse

- Density $> 30\%$
- Many non-zero updates
- Need fast random access
- Small matrices ($< 1000 \times 1000$)

Slide 375: Code Example: Sparse Matrices I

```python
import numpy as np
from scipy.sparse import random
from scipy.sparse.linalg import spsolve

# Create sparse matrix (5% density)
sparse = random(1000, 1000, density=0.05, format='csr')
print(f"Shape: {sparse.shape}")
print(f"Non-zero entries: {sparse.nnz}")
print(f"Memory: {sparse.data.nbytes / 1024:.1f} KB")

# Solve sparse system
b = np.random.randn(1000)
x = spsolve(sparse + sparse.eye(1000), b)
print(f"Solution norm: {np.linalg.norm(x):.4f}")
```

Slide 375: Code Example: Sparse Matrices II



Slide 376: Why Visualize Sparsity?

- Understand pattern of non-zeros
- Detect problems (e.g., banded, block)
- Debug matrix construction
- Present results in papers

Slide 377: The spy() Function

```
from matplotlib import pyplot as plt

plt.spy(sparse_matrix, markersize=1)
plt.title('Sparsity Pattern')
plt.show()
```

- Shows non-zero locations as dots
- White space = zeros

Slide 378: Basic spy() Example

```
import matplotlib.pyplot as plt
from scipy.sparse import random

sparse = random(100, 100, density=0.1, format='csr')
plt.spy(sparse, markersize=1)
plt.title('Sparsity Pattern (10% density)')
plt.show()
```

Slide 379: Customizing spy()

- `markersize` → dot size
- `marker='s'` → square markers
- `color='red'` → dot color
- `aspect='equal'` → square aspect ratio

Slide 380: Visualizing Different Patterns

- Random: scattered dots
- Diagonal: line along diagonal
- Banded: diagonal band
- Block diagonal: square blocks
- Tridiagonal: three diagonals

Slide 381: Creating Structured Sparse Matrices

```
from scipy.sparse import diags, block_diag

# Tridiagonal
diagonals = [np.ones(100), -2*np.ones(100), np.ones(100)]
offsets = [-1, 0, 1]
tridiag = diags(diagonals, offsets, format='csr')

# Block diagonal
block1 = random(10, 10, density=0.3)
block2 = random(10, 10, density=0.3)
blocks = block_diag([block1, block2])
```

Slide 382: Heatmap of Sparse Matrix

- Convert to dense (only if small!)
- `plt.imshow(sparse.toarray(), cmap='hot')`
- Shows values, not just presence
- **Not for large matrices**

Slide 383: Comparing Densities Visually

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, dens, title in zip(axes, [0.01, 0.05, 0.1], ['1%', '5%'
    , '10%']):
    mat = random(100, 100, density=dens)
    ax.spy(mat, markersize=1)
    ax.set_title(f'{title} density')
plt.tight_layout()
```

Slide 384: Analyzing Sparsity Structure

- Check if matrix is diagonal dominant
- Identify disconnected components
- Detect fill-in during operations
- Guide choice of solver (direct vs iterative)

Slide 385: Code Example: Sparsity Visualization I

```
1  import matplotlib.pyplot as plt
2  from scipy.sparse import diags, random, block_diag
3  import numpy as np
4
5  fig, axes = plt.subplots(2, 2, figsize=(10, 10))
6
7  # Random
8  axes[0,0].spy(random(50, 50, density=0.1), markersize=2)
9  axes[0,0].set_title('Random')
10
11 # Tridiagonal
12 diags_arr = [np.ones(50), -2*np.ones(50), np.ones(50)]
13 tridiag = diags(diags_arr, [-1,0,1])
14 axes[0,1].spy(tridiag, markersize=3)
```

Slide 385: Code Example: Sparsity Visualization II

```
15 axes[0,1].set_title('Tridiagonal')
16
17 # Block diagonal
18 b1 = random(30, 30, density=0.2)
19 b2 = random(20, 20, density=0.2)
20 blocks = block_diag([b1, b2])
21 axes[1,0].spy(blocks, markersize=2)
22 axes[1,0].set_title('Block Diagonal')
23
24 # Combination
25 combined = tridiag + random(50, 50, density=0.02)
26 axes[1,1].spy(combined, markersize=2)
27 axes[1,1].set_title('Tridiagonal + Noise')
28
```

Slide 385: Code Example: Sparsity Visualization III

```
29 plt.tight_layout()
30 plt.show()
```

Slide 386: Images as NumPy Arrays

- Image = 3D array (height, width, channels)
- Grayscale: (height, width)
- RGB: (height, width, 3)
- RGBA: (height, width, 4)
- Values: 0 to 255 (uint8) or 0.0 to 1.0 (float)

Slide 387: Loading Images

```
from matplotlib import pyplot as plt

# Load image
img = plt.imread('image.jpg')
# Returns: 0-1 for PNG, 0-255 for JPG

# Display
plt.imshow(img)
plt.show()
```

Slide 388: Displaying Images

```
plt.imshow(img)
plt.axis('off')       # Hide axes
plt.colorbar()        # Show intensity scale
plt.title('My Image')
plt.show()
```

Slide 389: Image Properties

```
print(f"Shape: {img.shape}")
print(f"Data type: {img.dtype}")
print(f"Min: {img.min():.3f}, Max: {img.max():.3f}")
```

Slide 390: Grayscale Conversion

```
# Simple average
gray = img.mean(axis=2)

# Luminance (perceived brightness)
# Gray = 0.299*R + 0.587*G + 0.114*B
gray = 0.299 * img[:,:,0] + 0.587 * img[:,:,1] + 0.114 * img
    [:,:,2]

# Result is 2D array
print(gray.shape)  # (height, width)
```

Slide 391: Cropping Images

```
# Array slicing: [row_start:row_end, col_start:col_end]
cropped = img[50:200, 30:150]

# For color images
cropped = img[50:200, 30:150, :]
```

Slide 392: Resizing Images (NumPy)

```
# Simple: slice every nth pixel
small = img[::2, ::2]  # Half size (lossy)

# Better to use SciPy or scikit-image
```

Slide 393: Resizing with SciPy

```
from scipy.ndimage import zoom

# Half size
resized = zoom(img, (0.5, 0.5, 1))

# order=1: bilinear interpolation
# order=3: cubic interpolation
```

Slide 394: Rotating Images

```
from scipy.ndimage import rotate

# Rotate 45 degrees
rotated = rotate(img, angle=45, reshape=True)

# Crop to original size
rotated = rotate(img, angle=45, reshape=False)
```

Slide 395: Flipping Images

```
# Horizontal flip (left-right)
flipped_h = img[:, ::-1, :]

# Vertical flip (up-down)
flipped_v = img[::-1, :, :]

# Using NumPy functions
flipped_v = np.flipud(img)
flipped_h = np.fliplr(img)
```

Slide 396: Color Adjustments – Brightness

```
# Increase brightness
img_bright = img * 1.2

# Decrease brightness
img_dark = img * 0.8

# Clip to valid range
img_bright = np.clip(img_bright, 0, 1)
```

Slide 397: Color Adjustments – Contrast

```
mean = img.mean()
contrast_factor = 1.5
img_contrast = (img - mean) * contrast_factor + mean
img_contrast = np.clip(img_contrast, 0, 1)
```

Info

Higher contrast factor = more extreme values.

Slide 398: Color Adjustments – Gamma Correction

```
# gamma < 1: lighter, gamma > 1: darker
gamma = 0.5    # lighter
img_gamma = img ** gamma

gamma = 2.0    # darker
img_gamma = img ** gamma
```

Info

Nonlinear adjustment that preserves shadows and highlights.

Slide 399: Image Filtering – Gaussian Blur

```
from scipy.ndimage import gaussian_filter

# sigma controls blur amount
blurred = gaussian_filter(img, sigma=2)
```

Info

Larger sigma = more blur.

Slide 400: Image Filtering – Median Filter

```
from scipy.ndimage import median_filter

# size is the kernel size (odd number)
denoised = median_filter(img, size=3)
```

- Good for salt-and-pepper noise
- Preserves edges better than Gaussian

Slide 401: Image Filtering – Sharpening

```
from scipy.ndimage import gaussian_filter

# Original + (Original - Blurred)
blurred = gaussian_filter(img, sigma=2)
sharpened = img + (img - blurred)
sharpened = np.clip(sharpened, 0, 1)
```

Slide 402: Edge Detection – Sobel Filter

```
from scipy.ndimage import sobel

# Grayscale image required
edges_x = sobel(gray_img, axis=0)
edges_y = sobel(gray_img, axis=1)
edges = np.hypot(edges_x, edges_y)
```

Slide 403: Edge Detection – Laplacian

```
from scipy.ndimage import laplace

# Finds regions of rapid intensity change
edges = laplace(gray_img)
```

Slide 404: Thresholding

```
# Binary mask
threshold = 0.5
mask = gray_img > threshold

# Apply mask to original image
result = img * mask[:, :, np.newaxis]

# Creates binary (black/white) image
```

Slide 405: Morphological Operations – Erosion & Dilation

```
from scipy.ndimage import binary_erosion, binary_dilation

# Structuring element (3x3 square)
struct = np.ones((3, 3))

# Erosion: removes small objects
eroded = binary_erosion(binary_img, structure=struct)

# Dilation: fills small holes
dilated = binary_dilation(binary_img, structure=struct)
```

Slide 406: Morphological Operations – Opening & Closing

```
from scipy.ndimage import binary_opening, binary_closing

# Opening = erosion then dilation (removes noise)
opened = binary_opening(binary_img, structure=struct)

# Closing = dilation then erosion (fills holes)
closed = binary_closing(binary_img, structure=struct)
```

Slide 407: Saving Images

```
from matplotlib import pyplot as plt

# Simple save
plt.imsave('output.jpg', img)

# Note: scipy.io.imsave is deprecated
# Use imageio or PIL for more control
```

Slide 408: ImageIO for Saving

```
import imageio.v3 as iio

# Save as PNG
iio.imwrite('output.png', img)

# Save as JPG with quality
iio.imwrite('output.jpg', img, quality=95)
```

Slide 409: Performance Tips

- Work with smaller images for testing
- Use in-place operations when possible
- Avoid loops (use vectorized operations)
- Convert to float32 for large images

Slide 410: Code Example: Complete Image Processing Pipeline I

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter, sobel, median_filter

# Load image
img = plt.imread('sample.jpg')
if img.dtype == np.uint8:
    img = img / 255.0  # normalize to 0-1

# Convert to grayscale
gray = np.mean(img, axis=2)

# Denoise
denoised = median_filter(gray, size=3)
```

Slide 410: Code Example: Complete Image Processing Pipeline II

```
15
16 # Detect edges
17 edges_x = sobel(denoised, axis=0)
18 edges_y = sobel(denoised, axis=1)
19 edges = np.hypot(edges_x, edges_y)
20
21 # Threshold edges
22 edges_binary = edges > edges.mean()
23
24 # Display results
25 fig, axes = plt.subplots(1, 3, figsize=(12, 4))
26 axes[0].imshow(gray, cmap='gray')
```

Slide 410: Code Example: Complete Image Processing Pipeline III

```
27 axes[0].set_title('Original Grayscale')
28 axes[1].imshow(denoised, cmap='gray')
29 axes[1].set_title('Denoised')
30 axes[2].imshow(edges_binary, cmap='gray')
31 axes[2].set_title('Edges')
32 for ax in axes:
33     ax.axis('off')
34 plt.tight_layout()
35 plt.savefig('image_processing_result.png', dpi=150)
36 plt.show()
```

Slide 411: Week Recap – Day 1

- Python basics: variables, data structures, functions, OOP
- Foundation for all scientific computing
- Writing reusable, readable code

Slide 412: Week Recap – Day 2

- NumPy: fast array operations, broadcasting, aggregations
- The backbone of scientific Python
- Vectorization for performance

Slide 413: Week Recap – Day 3

- Pandas: data wrangling, analysis, group operations
- Working with real-world tabular data
- Missing data, merging, reshaping

Slide 414: Week Recap – Day 4

- Matplotlib: from basic to advanced visualization
- Customization, 3D plotting, best practices
- Telling stories with data

Slide 415: Week Recap – Day 5

- SciPy: advanced scientific computing
- Sparse matrices for big data
- Image processing with NumPy/SciPy

Slide 416: Key Takeaways

- Python is powerful for research
- Vectorization $>$ loops
- Libraries work together (NumPy $\rightarrow$ Pandas $\rightarrow$ Matplotlib)
- Clean code saves time
- Visualize to understand

Slide 417: Where to Go Next – Statistics

- `scipy.stats` → distributions, hypothesis tests
- `statsmodels` → regression, time series
- `pingouin` → user-friendly statistics

Slide 418: Where to Go Next – Machine Learning

- `scikit-learn` → ML algorithms
- `tensorflow` / `pytorch` → deep learning
- `xgboost` → gradient boosting

Slide 419: Where to Go Next – Advanced Visualization

- `seaborn` → statistical plots (built on matplotlib)
- `plotly` → interactive, web-based
- `bokeh` → dashboard-style plots

Slide 420: Where to Go Next – Geospatial

- `geopandas` → spatial data
- `cartopy` → maps
- `rasterio` → satellite imagery

Slide 421: Where to Go Next – Specialized

- `biopython` → bioinformatics
- `astropy` → astronomy
- `nilearn` → neuroimaging
- `opencv` → computer vision

Slide 422: Learning Resources

- Official documentation (excellent!)
- Stack Overflow (community Q&A)
- Kaggle (datasets and notebooks)
- YouTube channels: Corey Schafer, Sentdex

Slide 423: Books for Further Study

- "Python for Data Analysis" – Wes McKinney
- "Python Data Science Handbook" – Jake VanderPlas
- "Effective Python" – Brett Slatkin
- "Fluent Python" – Luciano Ramalho

Slide 424: Staying Connected

- GitHub (share code, collaborate)
- ResearchGate / LinkedIn (professional network)
- Local Python user groups
- Online courses (Coursera, edX, DataCamp)

Slide 425: Final Message

Keep practicing!

Work on your own research problems

Don't memorize – learn to search effectively

Help others (teaching reinforces learning)

Happy Coding!

Thank You!

Happy Coding!

Instructors: X

Email: abatea72@gmail.com

Course Materials: `https://abatea.github.io/python-basic/`